

AZURE SHARED RESPONSIBILITY

A Real-World Analysis Using Personal Home Lab Infrastructure

On-Premises · IaaS · PaaS — Mapping Real Infrastructure to the Shared Responsibility Model

Project Title	Azure Shared Responsibility — Real-World Home Lab Analysis
Date	September 2026
Analyst	Bryan Sykes
AZ-900 Domain	Cloud Concepts — Shared Responsibility Model
Real Infrastructure Used	sykes-ubuntu (on-premises), sykeslogstorage (PaaS), Azure VM (IaaS planned)
Business Problem	Understanding which security controls are your responsibility vs Microsoft's across different deployment models
Coverage	Business problem, responsibility matrix, key decisions, production scale considerations

Analyst: Bryan Sykes | Home Lab Portfolio Project

1. The Business Problem It Solves

One of the most common and costly security failures in cloud environments is misplaced responsibility. An organization assumes Microsoft is securing something that is actually their own responsibility, or they spend time and money securing something Microsoft already handles by default. This gap is directly responsible for the majority of cloud data breaches.

In a real federal IT or enterprise environment, not understanding the shared responsibility model creates three specific business risks:

- Compliance failures - Auditors under FedRAMP, NIST 800-53, and CMMC frameworks require clear documentation of who is responsible for each security control. "We thought Azure handled that" is not an acceptable audit response.
- Security gaps - If a team assumes Microsoft patches the OS on an Azure VM (they don't... that's IaaS, it's your responsibility), that VM will go unpatched and become a vulnerability.
- Wasted security spend - If a team builds elaborate controls around something Microsoft already secures by default (like physical datacenter security), they are spending engineering time solving a problem that doesn't exist.

This project solves that problem by mapping real, live infrastructure across three deployment models, on-premises, IaaS, and PaaS to a clear, color-coded responsibility matrix. The result is a reference document that answers the question every cloud team needs to answer before any deployment:

"Who is responsible for this? Microsoft or us?"

1.1 Real Infrastructure Context

Rather than using generic textbook examples, this analysis maps the shared responsibility model to actual infrastructure currently running in the home lab:

Resource	Type	Deployment Model
sykes-ubuntu (ASUS X550ZA)	Physical laptop - personal hardware	On-premises - analyst owns and manages everything
sykeslogstorage	Azure Blob Storage	PaaS - Microsoft manages infrastructure, analyst manages data and access
Azure VM (Terraform project)	Azure Virtual Machine	IaaS - Microsoft manages physical layer, analyst manages OS and above

2. Architecture — The Responsibility Stack

The shared responsibility model is best understood as a vertical stack of layers. From physical hardware at the bottom, to data and identity at the top. As the deployment model moves from on-premises to IaaS to PaaS, the line separating Microsoft's responsibility from the analyst's responsibility moves progressively higher up the stack.

2.1 The Responsibility Stack by Deployment Model

The table below uses color coding to show responsibility at a glance:

Microsoft	You (Analyst)	Shared	Legend
Microsoft fully owns and manages this layer	You fully own and manage this layer	Both parties contribute to this layer	

Responsibility Layer	On-Premises sykes-ubuntu	IaaS Azure VM	PaaS sykeslogstorage
Physical hardware	You	Microsoft	Microsoft
Physical network infrastructure	You	Microsoft	Microsoft
Datacenter physical security	You	Microsoft	Microsoft
Hypervisor	You	Microsoft	Microsoft
Operating system	You	You	Microsoft
Network controls (firewall/NSG)	You	You	Shared
Applications	You	You	You
Data	You	You	You
Identity & access management	You	You	You
Encryption configuration	You	Shared	Shared

2.2 The Key Principle

The pattern that emerges from the matrix reveals the core principle of cloud shared responsibility:

"The higher up the service model stack you go from on-premises to IaaS to PaaS, the more responsibility shifts to Microsoft and the less you manage, but you always remain responsible for your data and your identities regardless of the deployment model."

This principle holds true across all three deployment models in this analysis and is the single most tested concept in the AZ-900 Cloud Concepts domain.

3. Key Decisions and Why I Made Them

3.1 Using Real Infrastructure Instead of Generic Examples

Most shared responsibility analyses use fictional organizations or generic placeholder resources. This analysis deliberately uses the actual infrastructure running in the home lab, sykes-ubuntu, sykeslogstorage, and a planned Azure VM. This decision was intentional for three reasons:

- It forces genuine understanding rather than pattern matching. You can't fake your way through a matrix built on infrastructure you actually operate

- It surfaces real-world nuance the sykeslogstorage analysis revealed a genuine security gap (TLS 1.0 default) that was discovered and fixed during review
- It creates a stronger portfolio artifact. A hiring manager reviewing this can see that the analyst understands responsibility boundaries for infrastructure they have actually built and administered

3.2 Three-Column Structure: (On-Premises, IaaS, PaaS)

The decision to include on-premises as a column alongside IaaS and PaaS was deliberate. Most cloud-focused analyses skip on-premises entirely. Including it makes the value proposition of cloud computing immediately visible you can see exactly which responsibilities disappear when you move a workload from a physical machine in your room to an Azure VM, and which ones remain.

For federal IT environments specifically, many agencies are in a hybrid state. Running some workloads on-premises and migrating others to Azure. Understanding the responsibility split across all three models is operationally critical, not just academically interesting.

3.3 The Three Cells That Required the Most Reasoning

Encryption Configuration: Shared in IaaS and PaaS

This is the most commonly misunderstood cell. The instinct is to mark encryption as "You" in IaaS because you control the VM. The correct answer is Shared because:

- Microsoft provides encryption at rest by default on Azure managed disks you did not configure this, it simply exists
- You control encryption configuration choices whether to use Microsoft-managed keys or customer-managed keys, TLS version enforcement, application-level encryption
- Because both parties contribute meaningfully, it is Shared

Memory hook: "Microsoft encrypts the floor, you lock the door."

Network Controls — Shared in PaaS

The instinct is to mark network controls as "You" in PaaS because you configure the firewall rules and private endpoints. The correct answer is Shared because:

- Microsoft manages the underlying network infrastructure physical switches, platform-level DDoS protection, routing between datacenters
- You configure access controls firewall rules, private endpoints, IP restrictions, RBAC-based network access
- Because both parties contribute at different layers, it is Shared

Memory hook: "Microsoft owns the pipes, you control the valves."

Datacenter Physical Security: Microsoft in PaaS

Once any workload moves into the cloud whether IaaS or PaaS physical datacenter security is entirely Microsoft's responsibility. You cannot visit, audit, or influence Azure's physical security controls. The security of the building, the guards, the access controls, and the hardware are completely abstracted away.

Memory hook: "If you can't touch it, it's not yours."

3.4 Real Security Gap Identified During This Analysis

During the portal review that accompanied this analysis, the sykeslogstorage storage account was found to have Minimum TLS version set to Version 1.0, the default when a storage account is created via CLI without explicitly specifying a TLS minimum. TLS 1.0 and 1.1 were retired by Microsoft on February 3, 2026.

This gap was identified precisely because this analysis forced a systematic review of which security controls fall under the analyst's responsibility. The fix was applied immediately via Azure CLI:

```
az storage account update --name sykeslogstorage --resource-group LinuxLogsRG --min-tls-version TLS1_2
```

This is the practical value of the shared responsibility model — it is not just an exam topic. It is a structured way to audit your own security posture by systematically asking "who is responsible for this?" for every layer of your infrastructure.

4. What I Would Do Differently at Production Scale

4.1 Formalize as a Living Document in Azure Policy

At home lab scale, this analysis lives in a Word document. At production scale, particularly in a federal IT environment subject to FedRAMP or CMMC compliance. The shared responsibility model needs to be operationalized, not just documented. This means:

- Azure Policy assignments that automatically enforce the controls that fall under the analyst's responsibility, TLS minimums, allowed locations, required tags, encryption settings
- Azure Security Center / Defender for Cloud running continuous compliance assessment against a defined baseline, automatically flagging when a resource drifts out of the expected security posture
- Formal System Security Plan (SSP) documentation mapping each NIST 800-53 control to either Microsoft's responsibility or the organization's responsibility required for FedRAMP authorization

4.2 Customer-Managed Keys for Encryption

In this home lab, Microsoft-managed encryption keys are used for sykeslogstorage. This is the default and is appropriate for non-sensitive log data. At production scale, particularly for data subject to federal data handling requirements, customer-managed keys (CMK) stored in Azure Key Vault would be required. This shifts encryption key custody from Microsoft to the organization — important for both compliance and supply chain risk management.

4.3 Private Endpoints for All PaaS Resources

sykeslogstorage currently accepts connections from the public internet (secured by RBAC and TLS, but still publicly reachable). At production scale, every PaaS resource would be deployed with a Private Endpoint, making it reachable only from within a virtual network and invisible to the public internet entirely. This is the network controls "Shared" cell in the matrix made fully secure: instead of relying on authentication to stop unauthorized access, the resource is not reachable at all from outside the VNet.

4.4 Identity — Zero Trust Architecture

Data and identity are always the analyst's responsibility regardless of deployment model. At home lab scale, RBAC role assignments and SSH key authentication represent the identity controls in place. At production scale in a federal environment, identity controls would need to meet Zero Trust requirements:

- Conditional Access policies - MFA required for all users, block legacy authentication protocols
- Privileged Identity Management (PIM) - Just-in-time elevation for administrative roles, no standing admin access
- Azure AD Identity Protection - Risk-based conditional access that automatically responds to suspicious sign-in patterns
- Separate identities for human users and service principals, no shared credentials

4.5 Documented Responsibility Assignment Matrix (RACI)

The matrix in this document is a personal reference artifact. At production scale, a formal RACI (Responsible, Accountable, Consulted, Informed) matrix would replace it, assigning each responsibility layer not just to "Microsoft" or "You" but to specific teams, roles, and individuals within the organization. In a federal IT context this would include the Authorizing Official (AO), System Owner, Information System Security Officer (ISSO), and Cloud Service Provider (Microsoft) as named parties with documented accountability.

5. Key Lessons

- Data and identity are always your responsibility, regardless of whether the deployment model is on-premises, IaaS, or PaaS. These two layers never shift to Microsoft.
- The shared responsibility model is a security audit tool, not just an exam topic. Systematically applying it to real infrastructure uncovered a genuine TLS misconfiguration that was immediately remediated.
- Encryption is almost always Shared in cloud deployments. Microsoft provides default encryption at rest and in transit, but configuration choices (key management, TLS version, application-level encryption) remain the customer's responsibility.
- Physical responsibility is entirely Microsoft's once you enter the cloud, IaaS and PaaS alike. If you cannot physically touch it, you are not responsible for its physical security.
- Network controls in PaaS are Shared. Microsoft owns the underlying infrastructure and provides platform-level protections, but configuration of access rules, private endpoints, and firewall policies is the customer's responsibility.
- The OS boundary is the key IaaS distinction. In IaaS, you manage the OS and everything above it. In PaaS the OS disappears from your responsibility entirely. This single boundary is what most IaaS vs PaaS exam questions are actually testing.

Documentation prepared by: Bryan Sykes | September 2026 | Azure Shared Responsibility — Real-World Home Lab Analysis